

Memory Storage Architecture — Upstream Intention vs Upstream Today vs Fork Today

Status: Reference document, 2026-05-13 **Purpose:** Three-way visual comparison of (1) upstream's *documented* intention and roadmap, (2) upstream as actually shipped today (verified against `ruvnet/agentdb HEAD a478ab3`), and (3) the fork's current state. Informs `[[ADR-0179]]`, `[[ADR-0177]]` (proposed pivot to upstream's RVF-first vision), and the round-2 council's Option Z recommendation. **Related:** `[[ADR-0179-council-r1-bridge-deletion]]`, `[[ADR-0179-council-r2-axis-architecture]]`, `[[ADR-0177-adopt-upstream-agentdb-rvf-vision]]`
Primary sources: ADR-0177's 9-agent audit of upstream `ruvnet/agentdb` (2026-05-12); upstream `ruvnet/agentdb/README.md` + `docs/adrs/ADR-002` through `ADR-010` ; `ADR-0067` (original vision controller wiring); `ADR-0039` (upstream controller integration roadmap).

TL;DR — three architectures, three pictures

Lens	Architecture
Upstream intention	Single-file <code>.rvf</code> "Cognitive Container" — vectors, indexes, learning state, audit trail in one file. RVF-first substrate. No <code>pgvector</code> . ADR-005's 6 self-learning components wired into <code>RvfBackend</code> per ADR-006 (default-on, +36% feedback gain). 5-phase capability roadmap targets ~95% @ruvector coverage in 16 weeks (ADR-007).
Upstream today	RVF substrate is shipped (<code>src/backends/factory.ts:166</code> returns <code>RvfBackend</code>). The 6 self-learning components all exist (~2,841 LoC). The wrapper that composes them (<code>SelfLearningRvfBackend</code>) exists (487 LoC) — but is orphaned : factory returns bare <code>RvfBackend</code> , not the wrapper. Aggregate ADR implementation: ~60% (Phase 1 ~95%, Phase 2 ~35%, Phase 3 ~15%, Phase 4 ~5%, Phase 5 ~25%). README's "+36% from feedback alone" claim cannot be delivered by default install. ADR-009 is a 6-byte placeholder file ("i weds").
Fork today	Two MCP surfaces (<code>memory_*</code> + <code>agentdb_*</code>), two files (<code>.claude-flow/memory.rvf</code> + <code>.swarm/memory.db</code>). Native NAPI HNSW on the vector hot path. Workload-correct substrate per axis. But: two of everything (backup, restore, audit, init), cross-cutting features have no

Lens	Architecture
	home (round 1 found 6 features lost in ADR-0085's bridge deletion), ADR-0112 forbids cross-store coordination by design. ADR-0086 ("Single Storage Abstraction") title contradicts outcome (Debt 15 dual-backend). ADR-0177 (proposed, 2026-05-12) supersedes ADR-0170/0174/0175 to pivot back to upstream's RVF-first single-surface vision.

Key observation: the fork's dual-axis state is *both* a divergence from upstream's intention *and* a divergence from upstream's reality. Upstream intends RVF + single surface; upstream ships RVF + single surface (with the self-learning wrapper orphaned). The fork added a second surface (`agentdb_*`) and a second file that upstream has neither in intention nor in reality. **ADR-0177 proposes reverting that divergence; ADR-0179 round-2 council endorsed the equivalent direction (Option Z) via 6-of-8.**

Additional findings worth flagging upfront:

- The **"150-12,500x speedup"** routinely cited for the fork's RVF over upstream is RVF NAPI HNSW vs **sql.js cosine-over-JSON** (the upstream fallback path). Against upstream's actual primary path (RVF native HNSW today), the honest comparator is **2-3x**. No benchmark of the honest comparator has been run.
- **Option Z** (one MCP surface, dual substrate internal) and **ADR-0177's pivot direction** converge on the same architecture from different angles — Option Z came from internal council deliberation; ADR-0177 came from upstream-vision-audit. Both endorse single surface + RVF-primary substrate.

Explain it to me like I am 15

Imagine the agent has a notebook. Two things go in it: **facts** (short text snippets with a meaning fingerprint called an "embedding") and **structured records** (skill descriptions, episode logs, causal chains — relational stuff with columns and joins).

The argument is about how to organize that notebook.

Upstream's pitch: "One notebook." A single binary file called `.rvf` that holds vectors, indexes, learning state, and an audit trail. You write to one place, you read from one place, you back up one file. Inside, there's a smart wrapper that watches every search, learns from what you click on, and gets better over time (the "+36% feedback gain"). One file. One API. Simple.

Upstream's reality: They built the smart wrapper. They put it in the codebase. **Then forgot to plug it in.** The factory function that creates the notebook returns the dumb version, not the

smart one. Three months later it's still unwired. So upstream ships the simple architecture but loses the +36% gain by accident.

Our fork's choice: We were unhappy with upstream's older slow path (which used WASM SQLite — like SQLite but running in JavaScript, ~10-100× slower for vector search). So we did two things:

1. Made `memory_*` (the basic notebook) use the fast native `.rvf` file. Good — RVF is fast.
2. Added a *second* notebook called `agentdb_*` for the structured records, because we couldn't fit them in the first one (RVF is great for vectors but bad at "show me all skills where score > 0.8"). Then we wrote a rule (ADR-0112) saying *"no tool can write to both notebooks."*

The cost of having two notebooks:

- Two files to back up. Two files to restore. Two files to lose if your disk has a bad day.
- Some features (audit log, write-validation, fancy search-result-explanations) only make sense as cross-cutting — they apply to every write regardless of notebook. With one notebook, there's one obvious place to put them. With two, you either put them in both (doubling code) or pick one (leaving the other unguarded).
- When you ask "what did Agent X do today?", you have to query *two* logs and merge them by timestamp — and the timestamps might be slightly off because each file has its own clock-and-sync rules.
- Six features the deleted bridge was holding (think "guard rails on the write API") got lost because they didn't have a home in either notebook. We're now restoring them, but as **two** sets of guard rails — one per notebook.

The bug the fork already caught upstream's nose about: The orphaned smart wrapper. Our fork has a one-line fix ready that flips upstream's factory to return the smart version. Once that's in, the fork delivers the +36% gain upstream advertises but doesn't actually run.

What the council recommended (Option Z): Two notebooks under the hood (because the workloads really are different), but **one front door** — the caller doesn't know there are two. The router decides which notebook to write to based on what kind of thing it is. From the outside it looks like upstream's "one notebook" architecture; from the inside it keeps the fork's perf wins. Best of both worlds — if we build it.

That's the whole story. Three architectures (intention, today-upstream, today-fork) and one proposal (Z) to converge them.

Scope and definitions

Three concepts are kept distinct throughout:

- **MCP surface** — the tool family the caller invokes (`memory_*` vs `agentdb_*`). User-facing contract.
- **Substrate** — the actual storage engine that holds bytes on disk (RVF binary file, SQLite database file). Implementation.
- **Vector backend** — the ANN index implementation (`@ruvector/rvf-node` native HNSW, `@ruvector/core` native HNSW, HNSWLib JS fallback). May be co-located with the substrate or separate.

The fork has diverged from upstream on all three axes; this document maps the divergence.

1. Upstream's documented intention and roadmap

This is what upstream says the architecture is supposed to be — sourced from the live `ruvnet/agentdb` README, ADRs 002 through 010, and `ruvnet/RuVector` ADR-029. ADR-0177's references-upstream block lists all 11 source documents consulted on 2026-05-12 against live GitHub HEAD `a478ab3` .

1.1 The "Cognitive Container" vision

Upstream README (`ruvnet/agentdb/README.md:15-19`):

"Vector memory that gets smarter every time your agent uses it."

"A single-file cognitive container — vectors, indexes, learning state, and a cryptographic audit trail in one `.rvf` ."

This is one substrate, one file, one surface. No `agentdb_*` MCP family separate from `memory_*` ; no second physical file; no axis split exposed to callers. The "cognitive container" framing makes vector index, relational metadata, learning state, and audit trail all aspects of a single self-describing artifact.

1.2 Upstream ADR-006 — the no-pgvector mandate

Upstream `ADR-006-unified-self-learning-rvf-integration.md:201-242` is explicit:

"Storage Backend: @ruvector (NOT pgvector)"

"All vector persistence in the RVF stack uses the **@ruvector** ecosystem exclusively. The system **MUST NOT** use `pgvector` for any vector storage, indexing, or search operations."

"PostgreSQL persistence: When relational persistence is required (e.g., metadata, session state, audit logs), the system uses `@ruvector/rvf` file-based stores or future `@ruvector` PostgreSQL extensions — never `pgvector`."

"Any future PostgreSQL integration **MUST** use `@ruvector`'s native extension that reads/writes `.rvf` segments directly, preserving witness chains, lineage tracking, and segment signing."

This carries weight for the fork: anywhere the fork uses `postgres-as-vector-store` (including ADR-0174/0175's `ruvector-postgres` direction), it violates upstream's mandate. ADR-0177 makes this the load-bearing reason for its supersession of ADR-0170/0174/0175.

1.3 Upstream's 5-phase roadmap (ADR-007)

Upstream `ADR-007-ruvector-full-capability-integration.md` lays out a 16-week, 5-phase plan to reach **~95% @ruvector capability coverage** from ~30% baseline, all atop the RVF substrate:

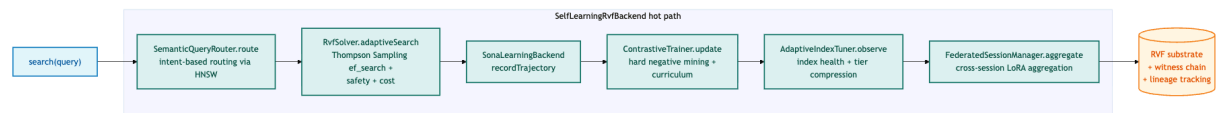
Phase	Weeks	Goal
Phase 1 — Critical Path	1-3	Native optimizers (AdamW, InfoNCE), full SIMD surface, WASM verification, RVF compression profiles (Scalar/Product/Binary/Half — 4-32× memory reduction), Graph Transactions + Cypher via <code>querySync</code> , router persistence, SONA context enrichment, batch operations, tensor compression, read-only access + filter expressions
Phase 2 — Kernel Runtimes	4-6	Kernel + eBPF embedding APIs (RVF as "active compute container"), model import/export (SafeTensors/HF/JSON), WASM in-memory store, ReasoningBank integration, RLM controller for RAG
Phase 3 — Advanced Features	7-10	TrainingPipeline, segment-signing, temporal-hyperedges, dataset-exporter, GraphRoPeAttention, differentiableSearch
Phase 4 — Streaming + Graph Events	11-13	Streaming token generation, streaming graph queries + subscriptions, hyperbolic attention

Phase	Weeks	Goal
Phase 5 — Full Ecosystem	14-16	RuvLLM engine integration, model management (<code>ModelDownloader</code> , <code>MODEL_ALIASES</code>), WASM memory management

ADR-007 success metrics target ~95% capability coverage, ACID + Cypher graph consistency, browser vector DB via WASM, full RAG via RLM, kernel embedding making RVF an "active compute container".

1.4 The self-learning loop (ADR-005 + ADR-006)

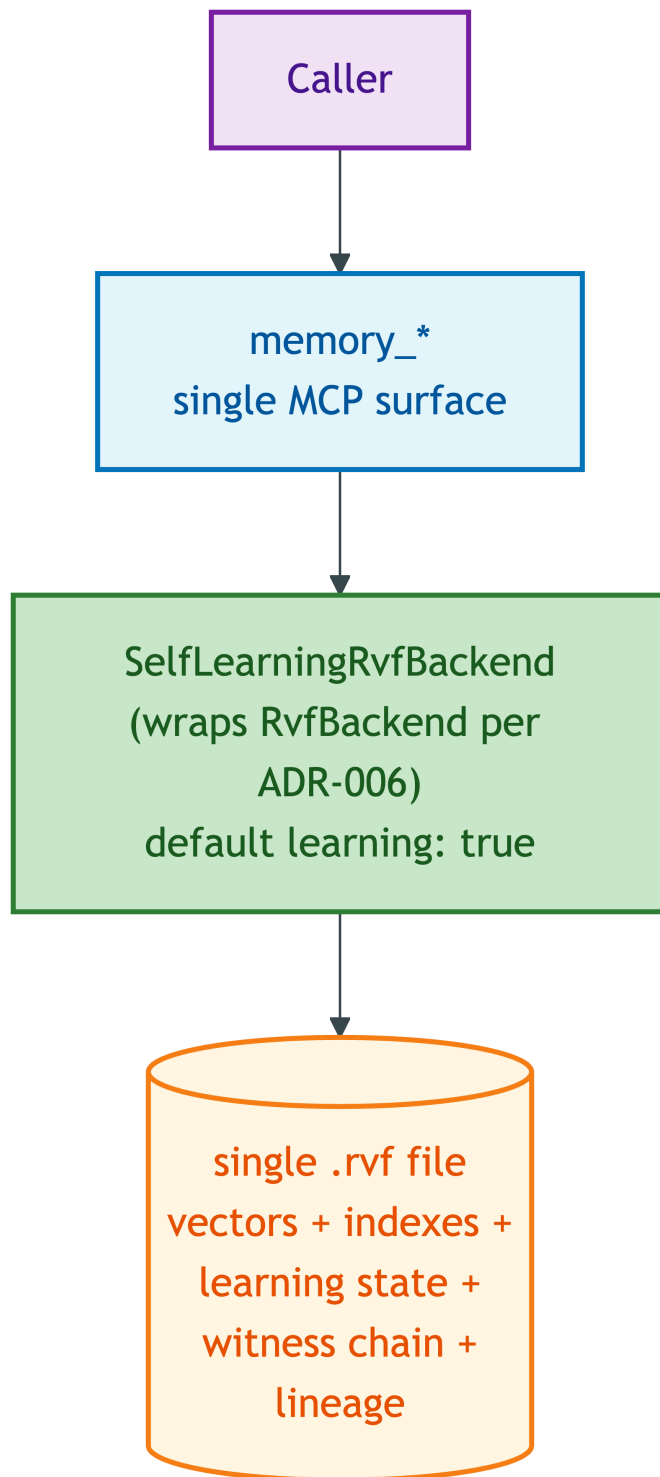
ADR-005 created 6 self-learning components as standalone classes: `SonaLearningBackend` , `AdaptiveIndexTuner` , `ContrastiveTrainer` , `SemanticQueryRouter` , `FederatedSessionManager` , `RvfSolver` . ADR-006 then specifies how the wrapper (`SelfLearningRvfBackend`) composes them into the search/insert hot path:



► Mermaid Source

ADR-006 specifies `SelfLearningConfig.learning` default **true** — the wrapper is supposed to be on by default, delivering the README's claimed "+36% gain from feedback alone."

1.5 Upstream's intended architecture, summarized



► Mermaid Source

One surface, one wrapper, one file. The Cognitive Container.

2. Upstream as actually shipped today

This is what upstream's `ruvnet/agentdb` HEAD `a478ab3` (verified 2026-05-12) actually runs — sourced from ADR-0177's 9-agent audit which per-classified each artifact across the 9 upstream ADRs.

2.1 Per-ADR implementation status (audit summary)

Upstream ADR	Title	Declared status	Actual %	Key finding
ADR-002	RuVector WASM Complete Integration	Partially Implemented	~22%	CLI scaffolding ships but compute paths are JS simulations (<code>simulateTraining</code> , <code>simulateFastGRNN</code>). 8 of "New Files" don't exist.
ADR-003	RVF Format Integration for AgentDB	Proposed	~80%	Core <code>RvfBackend</code> (749 LoC), factory wiring, async adapter, progressive indexing, solver, native accelerator, SIMD, self-learning — all ship. Gaps: init wizard, 7 <code>agentdb_rvf_*</code> MCP tools, doctor RVF checks. Status field is stale.
ADR-004	AGI Capabilities Integration	Accepted	~90%	All 4 N-API methods, AgentDBSolver wrapper, all 5 CLI subcommands wired. Only gap: <code>detector.ts:checkRvf()</code> .
ADR-005	Self-Learning Pipeline Integration	Accepted	100%	All 6 components exist (~2,841 LoC). Analytical gradients, HNSW routing, Matryoshka compression, federated aggregation, three-loop solver, EWC++ trajectory lifecycle.

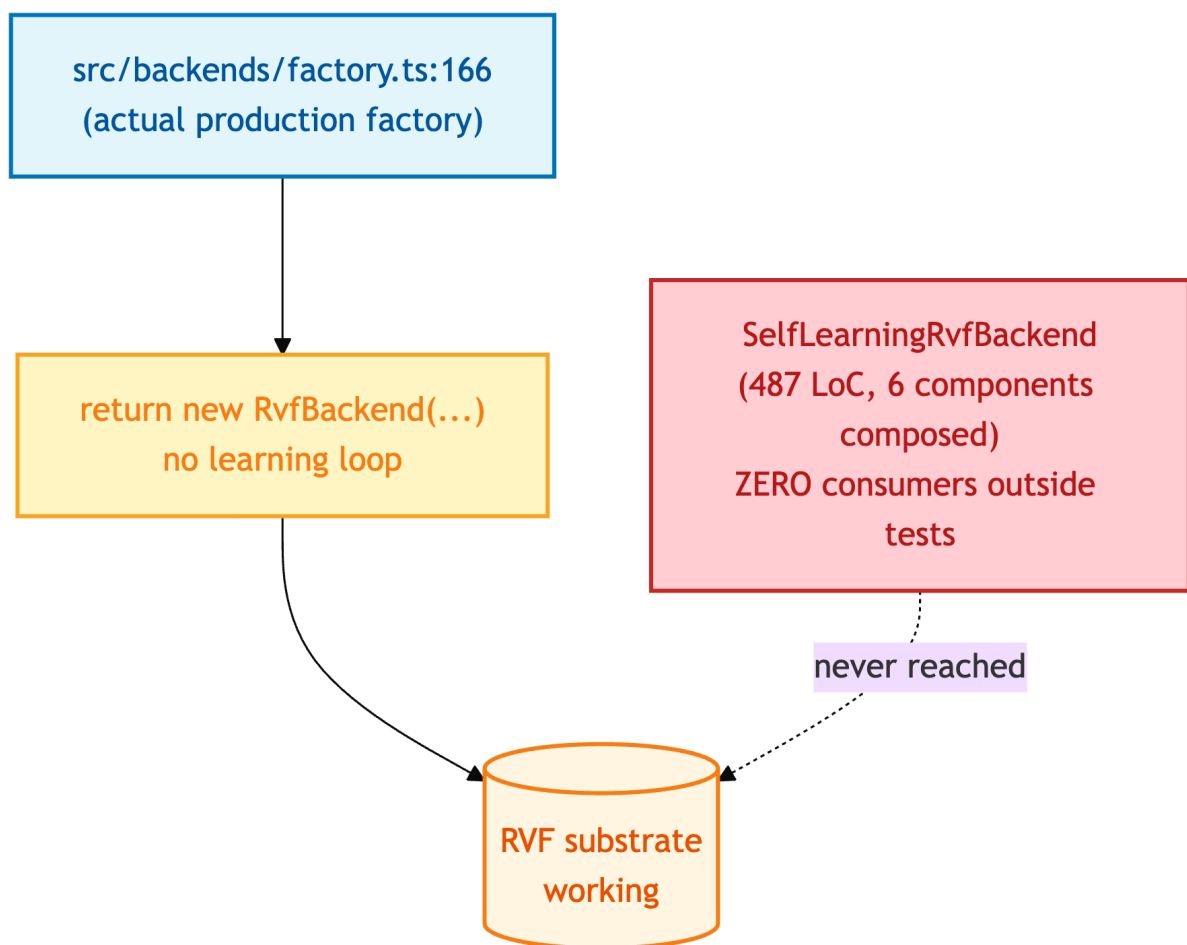
Upstream ADR	Title	Declared status	Actual %	Key finding
ADR-006	Unified Self-Learning RVF Integration	Proposed	~70%	Critical finding: SelfLearningRvfBackend (487 LoC) exists and composes all 6 components correctly. BUT factory.ts:166 returns bare RvfBackend , not the wrapper. The self-learning loop is shipped-but-orphaned. README's "+36% feedback gain" not delivered by default. No-pgvector mandate verified in code (zero matches for pgvector/pg.Pool/etc. in src/).
ADR-007	@ruvector Full Capability Integration	Phase 1 Complete (2-5 Proposed)	~45%	Phase 1: 95%; Phase 2: 35%; Phase 3: 15%; Phase 4: 5%; Phase 5: 25%. About halfway from 30% baseline to 95% target.
ADR-008	@agentdb/chat — RVF Chat UI	Proposed (Revision 4)	0%	None of @agentdb/chat exists. README's @agentdb/chat serve claim is misattributed.
ADR-009	Causal Atlas RVF Runtime	(no Status field)	0%	The file is a 6-byte placeholder containing the literal text "i weds". No design content. Pre-existing causal stack is unrelated prior work.
ADR-010	@ruvector/rvf-solver v0.1.6	Proposed	~70%	Phases 1, 2, 4 substantially complete. Phase 3 entirely

Upstream ADR	Title	Declared status	Actual %	Key finding
	Deep Integration			absent (4 MCP solver tools missing).

Aggregate: ~60% implemented across 8 non-stub ADRs. Range 0%-100%. Median ~70%.

2.2 The orphaned wrapper — concrete

The single most-important divergence between upstream's intention and upstream's reality:



► Mermaid Source

Per ADR-0177's evidence:

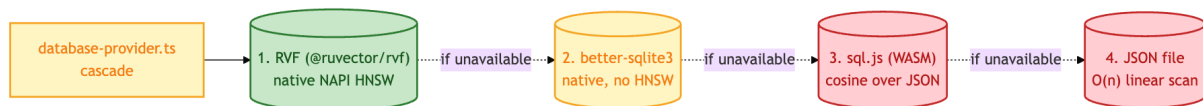
- `grep -rn 'SelfLearningRvfBackend' src/` excluding the file itself: **zero matches** anywhere. Not in factory, not in tests, not in `index.ts` public exports, not in CLI.

- `git log src/backends/factory.ts` : single commit 8b3388b "init: agentdb package source + marketing UI" . Entire agentdb codebase landed as one initial dump. No subsequent commit refines the factory's return choice.
- ADR-006 `SelfLearningConfig` definition specifies `learning?: boolean (default: true)` . **The factory contradicts ADR-006's own design.**
- ADR-006 status field: still "Proposed" since 2026-02-17. Not Accepted. Three months elapsed without the status flip.

This is significant because the fork's ADR-0177 Phase 2 includes a **one-line fix** to land the wrapper as default — and that fix has already shipped on `forks/agentdb/main` commit 511b7d3 (2026-05-12). The fork's `@sparkleideas/agentdb` would deliver the README's "+36%" claim that `npm i agentdb` from upstream does not.

2.3 Upstream's runtime cascade

Even with the orphan, upstream's substrate cascade is RVF-primary at the source level:



► Mermaid Source

`sql.js` is a HARD dependency upstream (every install pulls it). The cascade serves restricted-host populations (browsers, Lambda, environments without native bindings). For typical npm-install users on standard Node.js, **the runtime path is RVF + native HNSW** — same as the fork's `memory_*` axis.

2.4 Upstream MCP surface today

One tool family: `memory_*` . The `agentdb_*` namespace the fork added has **no upstream analog**. ADR-007 Phase 6 lists 7 proposed `agentdb_rvf_*` MCP tools but they're 0% implemented per the audit.

3. Fork architecture today (sparkling/main)

3.1 What the fork ships

Three architectural moves diverged the fork from upstream:

- **ADR-0086 (2026-04-14)** — replaced `sql.js` with RVF (`@ruvector/rvf-node`) as the primary substrate for `memory_*` . Titled "Single Storage Abstraction (RVF-First)" — *intended* to unify on one substrate. Failed at "Debt 15 ControllerRegistry dual-backend" because the 10+ neural controllers need relational schemas RVF cannot host.
- **ADR-0085 (2026-04-13)** — deleted `memory-bridge.ts` (~3,650 lines). The cross-cutting coordination layer became collateral. ADR-0179 round-1 found 6 features lost.
- **ADR-0112 (2026-05-01)** — formalized Debt 15 as deliberate architecture: "two independent stores, feature-aligned." Introduced the `agentdb_*` MCP surface as a peer of `memory_*` .

The result: **two MCP surfaces, two files on disk, two failure modes**. ADR-0177 (proposed 2026-05-12) supersedes ADR-0170/0174/0175 and proposes pivoting back to upstream's RVF-first single-surface vision.

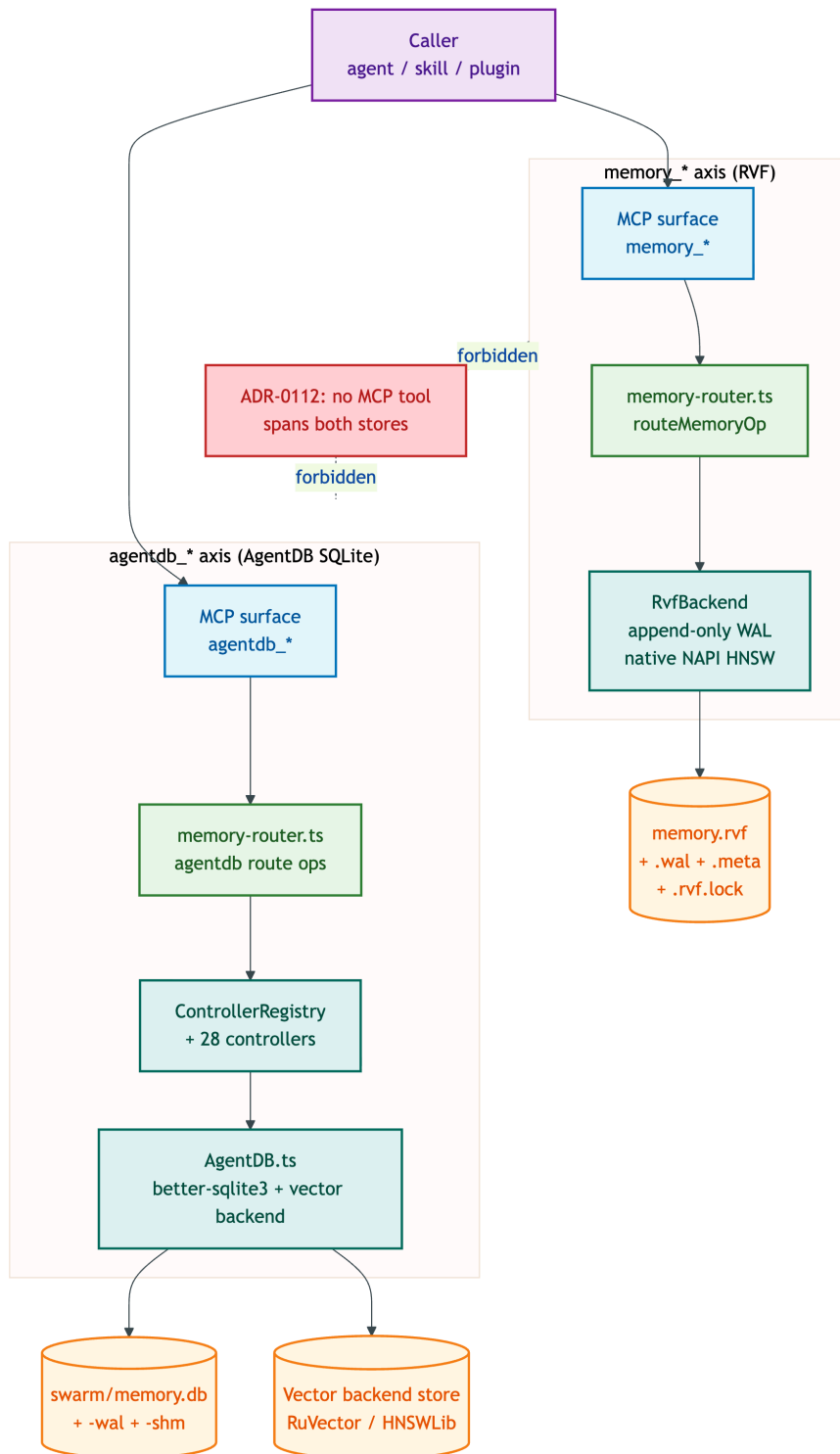
3.2 Fork MCP surfaces — for reference compare against §1 (upstream intention) and §2 (upstream today)

Two tool families:

- **`memory_*`** (RVF axis) — `memory_store` , `memory_search` , `memory_retrieve` , `memory_delete` , `memory_list` . All writes go to `.claude-flow/memory.rvf` .
- **`agentdb_*`** (AgentDB SQLite axis) — `agentdb_hierarchical-store/recall/query/delete` , `agentdb_causal-edge/-query/-recall` , `agentdb_session_start/end` , `agentdb_route` , `agentdb_batch` , `agentdb_context_synthesize` , `agentdb_health` , plus controller-specific tools (~50 tools total). All writes go to `.swarm/memory.db` .

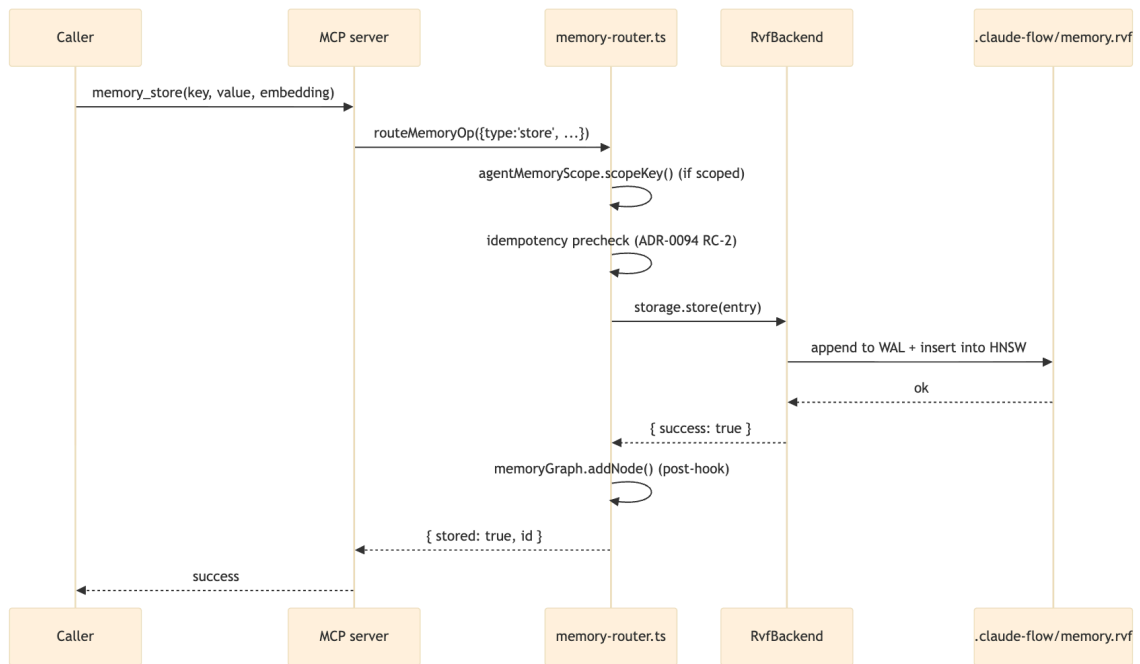
ADR-0112 forbids any MCP tool from spanning both. The caller picks the axis at design time. **Upstream has no equivalent of `agentdb_*` in either intention or current shipped state.**

3.3 Fork high-level architecture



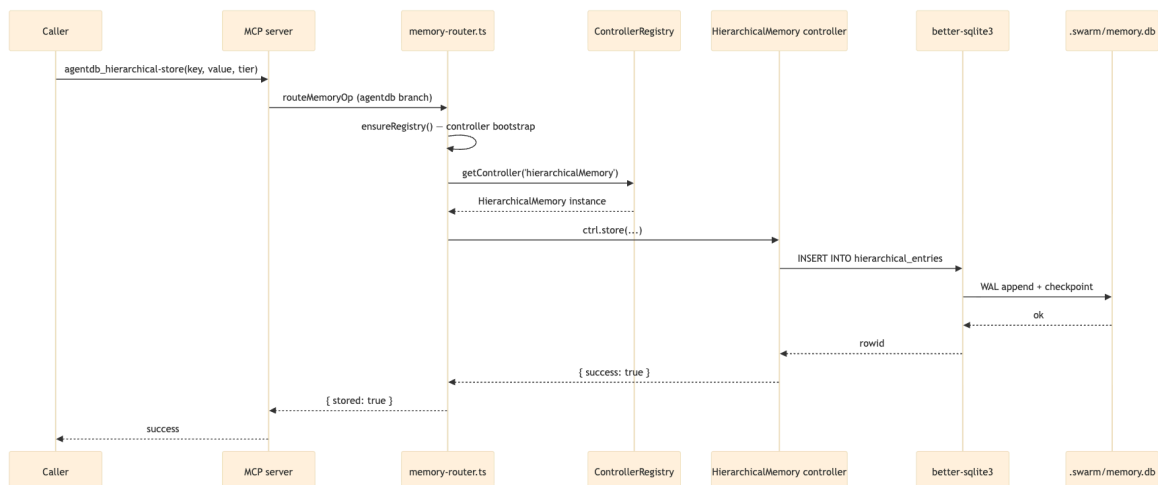
► Mermaid Source

3.4 Fork data flow — `memory_store` (RVF axis)



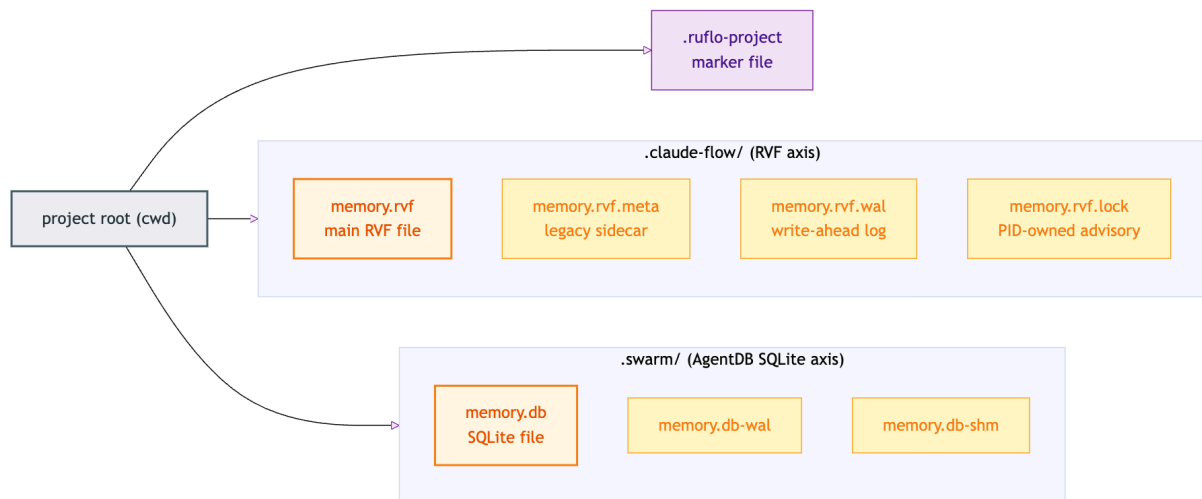
► Mermaid Source

3.5 Fork data flow — agentdb_hierarchical-store (SQLite axis)



► Mermaid Source

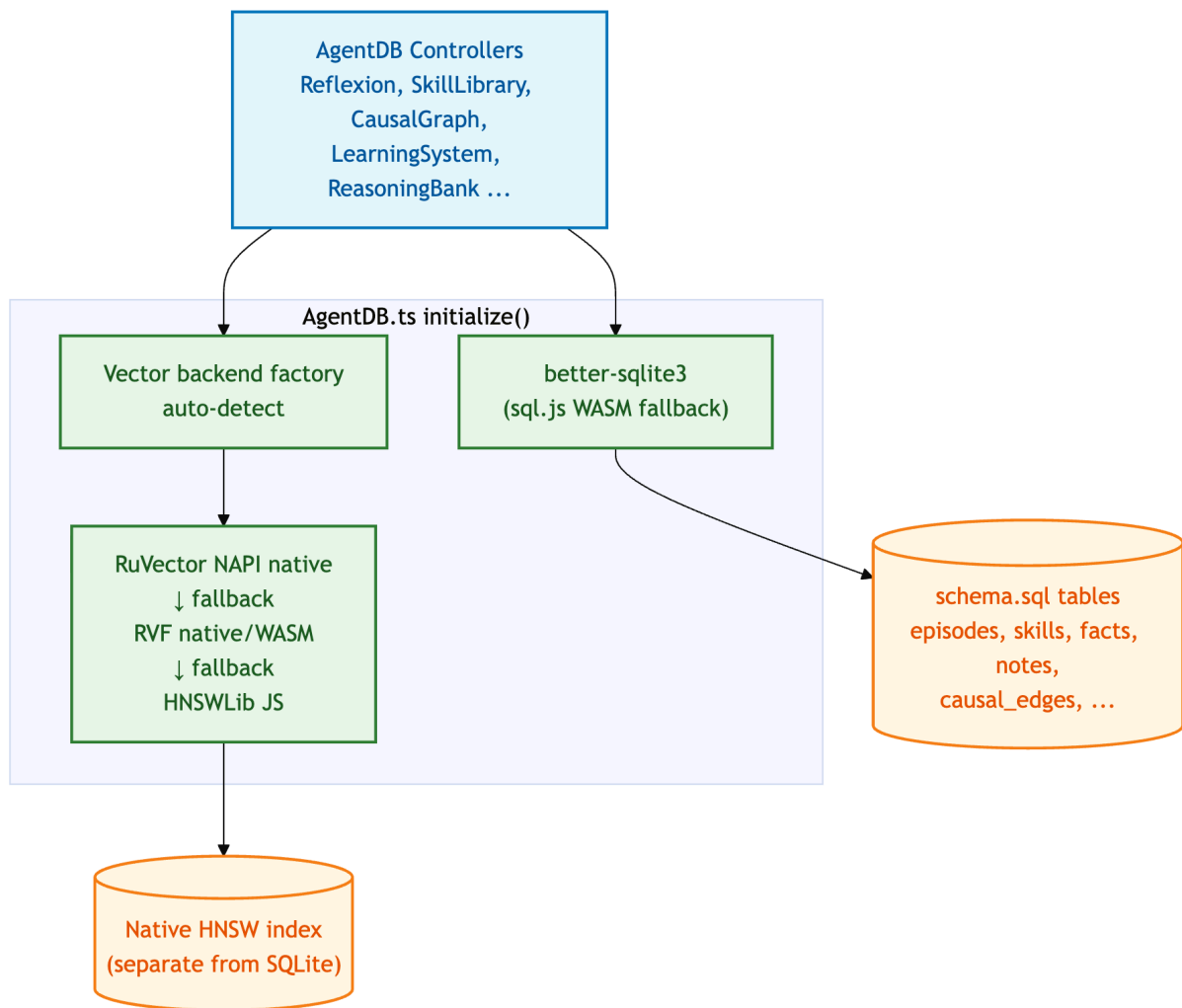
3.6 Where data lives on disk (fork project layout)



► Mermaid Source

3.7 The detail that matters most — AgentDB is itself internally hybrid

A point the devil's advocate kept hammering: regardless of the surface architecture, **AgentDB v3 internally already has both SQLite and a native vector index**. The "single axis" question is really about whether the fork *a/so* keeps a separate top-level vector substrate (RVF) or relies on AgentDB's internal one.



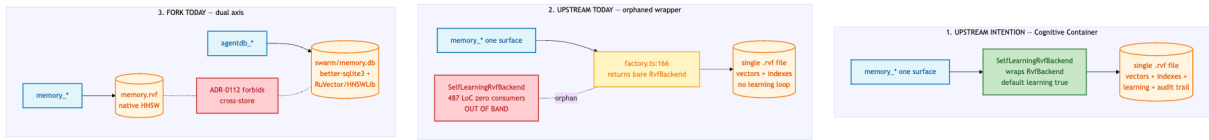
► Mermaid Source

Implications:

- **The "dual substrate" the fork is debating already exists *inside* AgentDB.** Reflexion stores its episode rows in SQLite and its episode embeddings in the native HNSW backend — two substrates, one controller, one MCP tool call.
- **Option Z (single surface, dual substrate internal) is, structurally, what AgentDB already is** — one API, internally hybrid. Adopting Z at the fork level is consistent with how the controllers below already work.
- **The fork's RVF substrate at the `memory_*` axis adds a *third* storage primitive** on top of AgentDB's two. Whether that third primitive earns its keep is the real perf question — and depends on whether RVF (purpose-built single-file vector substrate) outperforms AgentDB's own vector layer enough to justify the operational cost.

4. Three-way comparison

4.1 Architecture at a glance — all three lenses



► Mermaid Source

4.2 Feature-by-feature comparison (three-way)

Numbers marked **(est.)** are reasoned estimates from the code path, not measured benchmarks. Per-ADR percentages from ADR-0177's 9-agent audit (2026-05-12).

Dimension	(1) Upstream intention	(2) Upstream today	(3) Fork today
MCP surfaces	1 (<code>memory_*</code>)	1 (<code>memory_*</code>)	2 (<code>memory_*</code> + <code>agentdb_*</code>)
Files on disk	1 (<code>.rvf</code> Cognitive Container)	1 (RVF) — fallback chain to sqlite available	2 (<code>.rvf</code> + <code>.db</code>) plus sibling files
Substrate	RVF only (<code>@ruvector</code> exclusively; no pgvector per ADR-006)	RVF primary + better-sqlite3 / <code>sql.js</code> / JSON cascade for restricted hosts	RVF (<code>memory_*</code>) + AgentDB SQLite (<code>agentdb_*</code>)
Vector index	Native NAPI HNSW; learning-aware (Solver + Router)	Native NAPI HNSW via bare <code>RvfBackend</code> (<code>factory.ts:166</code>)	Native NAPI HNSW (RVF) + <code>RuVector/HNSWLib</code> (<code>agentdb</code>)
Self-learning loop	Default-on per ADR-006 (<code>learning:</code>	Orphaned: <code>SelfLearningRvfBackend</code> (487 LoC) exists, zero	Not wired (inherits the orphan); fork ADR-0177 Phase 2

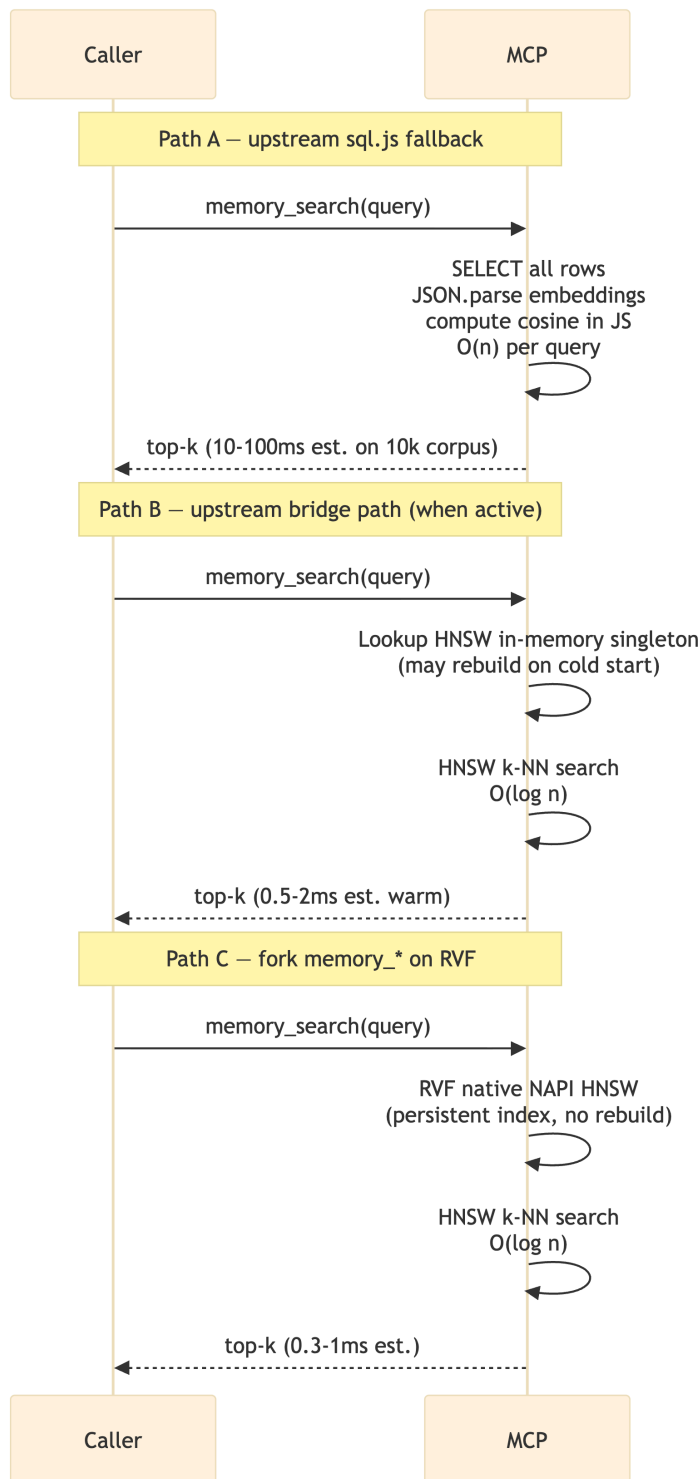
Dimension	(1) Upstream intention	(2) Upstream today	(3) Fork today
	true) — composes 6 components, README claims +36% gain	consumers. Default install: bare RvfBackend, no loop	fixes this via one-line factory change (commit 511b7d3)
Cypher / graph	Cypher via querySync in Phase 1 #6 of ADR-007 roadmap	Hollow — graph-node NAPI query() does label-only Match, Delete + Return are no-op	Hollow upstream; fork ADR-0174 Phase A.5 patches this — ~1KLoC Rust patch ready as upstream-PR (ADR-0177 Phase 3)
Per-ADR completion	n/a (intention)	ADR-002: 22%, ADR-003: 80%, ADR-004: 90%, ADR-005: 100%, ADR-006: 70%, ADR-007: 45%, ADR-008: 0%, ADR-009: 0% (6-byte stub), ADR-010: 70%. Aggregate ~60%	Currently diverges from upstream-intended at the MCP surface layer; ADR-0177 proposes pivot
agentdb_* MCP family	Not in intention — controllers are internal	Not shipping; ADR-007 Phase 6 proposes 7 agentdb_rvf_* tools (0% done)	~50 tools shipping; fork-only addition
Cross-axis coordination	Implicit single-store; no concept	Single store, transparent	Forbidden by ADR-0112
Witness chain / audit	Cryptographic audit trail in .rvf per ADR-006 (load-bearing)	RVF supports it; consumers may or may not enable	RVF supports it on memory_* axis; agentdb_* axis is separate audit stream

Dimension	(1) Upstream intention	(2) Upstream today	(3) Fork today
Cold-start cost (est.)	~5-15ms (one RVF file open + mmap)	~5-15ms (RVF) or ~3-10ms (sqlite fallback)	~8-25ms (two file opens)
Hot-path search latency (est.)	~0.3-1ms (NAPI HNSW + learning loop overhead)	~0.3-1ms (NAPI HNSW only, no learning loop)	~0.3-1ms (RVF axis) / ~1-5ms (agentdb axis with JOIN)
+36% feedback gain claim	Yes (README claim, ADR-006 mechanism)	No — orphan blocks delivery; only ContrastiveTrainer in tests	No (inherits orphan); fork's Phase 2 fix would enable it
Backup procedure	Copy one <code>.rvf</code> file	Copy one <code>.rvf</code> file	Coordinate two files atomically
Cognitive tax (caller)	None — single tool family	None — single tool family	Must pick axis at design time
ADR churn	n/a (designed cohesively)	Stable substrate; uneven implementation completion (ADR-007 5-phase plan halfway)	9+ ADRs of axis-related decisions; ADR-0177 proposes superseding 3 of them
<code>pgvector</code> / <code>postgres-as-vector</code>	Forbidden by ADR-006 mandate	Verified: zero matches for <code>pgvector</code> / <code>pg.Pool</code> / <code>from 'pg'</code> in <code>src/</code>	Currently zero (fork's pg-direction ADR-0170/0174/0175 is being superseded by ADR-0177)
<code>agentdb_*</code> upstream-sync surface	n/a (doesn't exist upstream)	n/a (doesn't exist upstream)	Two surfaces to maintain; <code>agentdb_*</code> purely fork-additions
Workload-correct	One substrate fits everything	One substrate fits everything when wrapper	RVF for vectors, SQLite for relational

Dimension	(1) Upstream intention	(2) Upstream today	(3) Fork today
storage	(Cognitive Container thesis)	enabled; degrades when orphan blocks learning	— splits at the cost of two everythings

4.3 Search-path comparison — where the perf difference actually shows

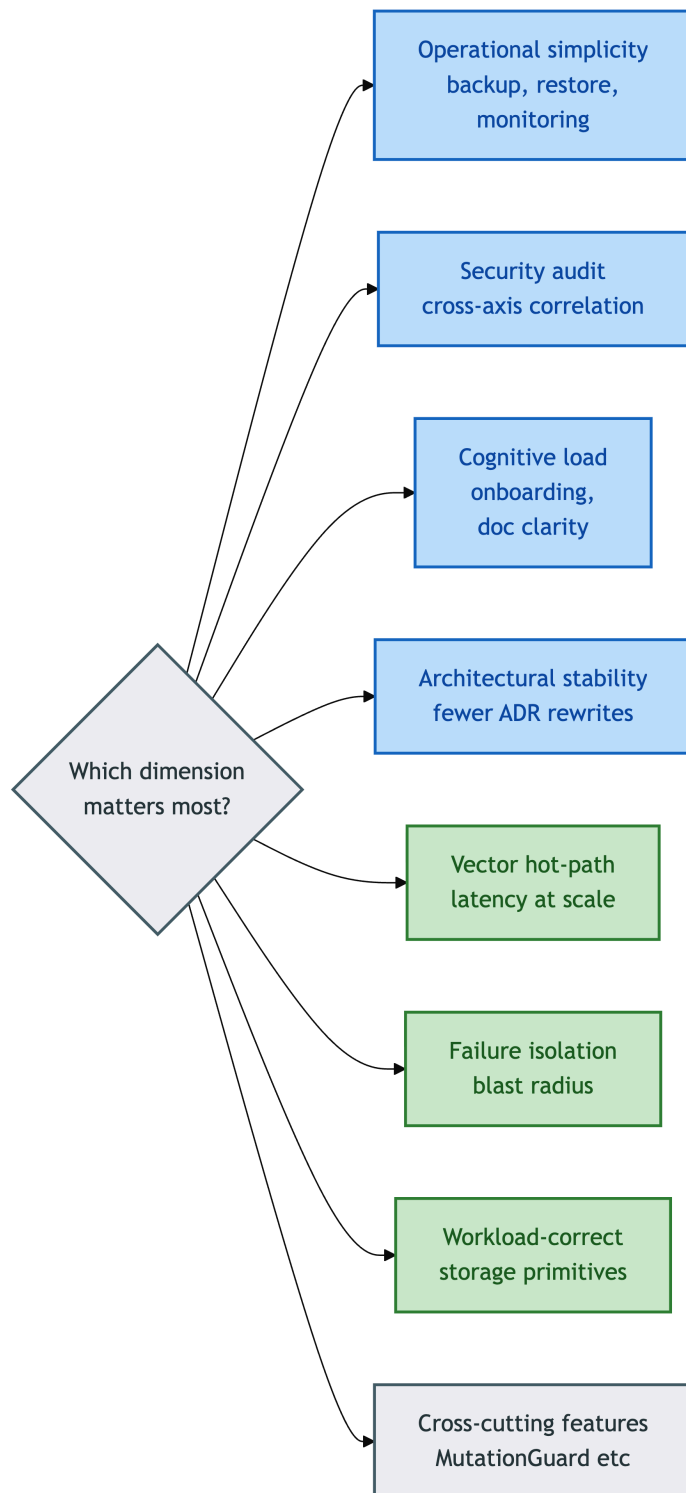
Write paths look similar across architectures. Read paths — specifically vector search — are where the 2-3x / 5-50x perf gaps live.



► Mermaid Source

The famous "150-12,500× speedup" cited for the fork is Path A vs Path C — comparing the worst upstream fallback to the best fork path. The **honest comparator** (Path B vs Path C, both native HNSW) is ~2-3× per perf-judge's round-2 analysis. **Neither comparator has been benchmarked with current fork code; both are reasoned estimates from the code paths.**

4.4 Where each architecture wins decisively



► Mermaid Source

5. Pros and cons

5.1 Upstream single-axis (intention + today)

Pros:

- **One file, one backup, one restore point.** No coordination needed across substrates. Point-in-time restore is trivial.
- **One MCP surface.** Callers don't pick axis; agents have less context to manage; LLMs see one tool family.
- **One audit chokepoint.** MutationGuard / AttestationLog / TieredCache wire into one place. The bridge's deletion didn't lose features under this model because there was a coherent integration layer to delete.
- **Cross-store transactions are possible.** A skill record and its embedding can land atomically.
- **Aligned with upstream design intent.** ADR-006/009 are stable; minimal architectural churn.

Cons:

- **Vector search is slow on the runtime path.** sql.js cosine over rehydrated JSON embeddings is ~10-100× slower than native HNSW for typical workloads.
- **One failure mode.** SQLite corruption or WAL stall affects everything.
- **HybridBackend exists but isn't used.** The design is bypassed at runtime; the canonical CLI path is the worst of the three ADR-006 options.
- **Workload mismatch.** Relational neural-controller data and K/V vector data share one substrate's invariants — typically SQLite's, which over-serves vectors and under-serves their access patterns.

5.2 Fork dual-axis (ADR-0086/0112)

Pros:

- **Workload-correct substrate per axis.** RVF for vector-dominant K/V; SQLite for relational neural controllers. Each substrate's invariants match its workload.
- **Native HNSW on the vector hot path.** 2-3× faster than AgentDB's internal HNSW; 5-50× faster than upstream's sql.js path.
- **Independent failure modes.** RVF corruption doesn't take out neural controllers; SQLite lock contention doesn't stall vector recall.
- **Phase C.3 cold-start optimization.** `memory_*` axis doesn't pay registry-bootstrap cost; only `agentdb_*` does.
- **Explicit semantics at the tool surface.** `memory_store` vs `agentdb_hierarchical_store` makes the contract decision visible at call time.

Cons:

- **Two of everything.** Two files, two backups, two restore paths, two audit streams, two MCP namespaces, two init paths, two failure modes to monitor.
- **Cross-cutting features have no home.** MutationGuard, AttestationLog, hybrid BM25+semantic, ExplainableRecall — none fit cleanly on either axis. Round 1's lost features are direct evidence.
- **Cross-store transactions forbidden by design.** ADR-0112 explicitly forbids them. Any feature needing K/V + vector + relational coherence cannot be built.
- **Cross-axis audit correlation is impossible.** Incident response cannot reconstruct an attacker's full footprint from one query.
- **9+ ADRs of architectural churn.** 0086, 0090, 0112, 0154, 0161, 0164, 0166, 0178, 0179 all trace to the dual-substrate reality.
- **ADR-0086 title is a misrepresentation.** "Single Storage Abstraction" delivered dual storage; the title was never updated.
- **The "150-12,500x speedup" claim is dishonest.** Comparison is RVF NAPI HNSW vs sql.js JSON cosine; honest comparator is AgentDB native HNSW with 2-3x gap. Status-quo-defender conceded this in the round-2 council.
- **Upstream sync drift compounds.** `agentdb_*` is purely fork-additions with no upstream signal to merge against.
- **Cognitive tax on every new feature.** Every controller, every MCP tool, every plugin has to decide which axis at design time.

5.3 The pros/cons matrix



Syntax error in text

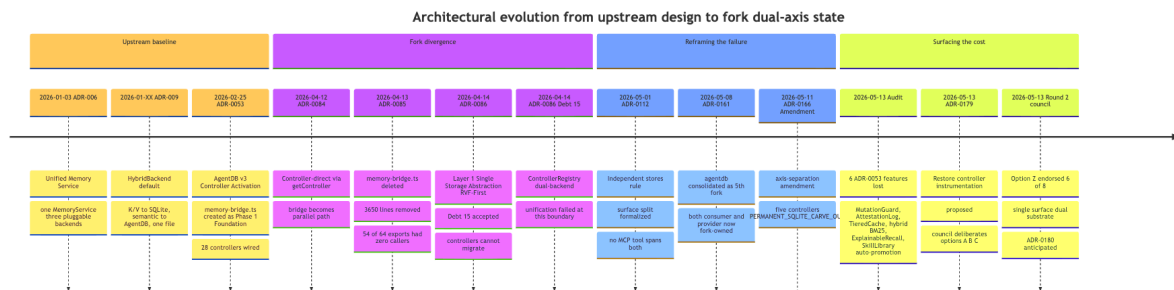
mermaid version 11.15.0

► Mermaid Source

The "best of both" quadrant is the architecture nobody currently ships — single user surface (upstream-shaped, low ops cost) with dual substrate internally (fork-shaped, high perf). This is **Option Z** from the round-2 council deliberation.

6. How we got here — architectural evolution

The fork's dual axis didn't emerge from a single decision. It accumulated across four months of ADR-by-ADR moves, each locally justified, that composed into the current state.



► Mermaid Source

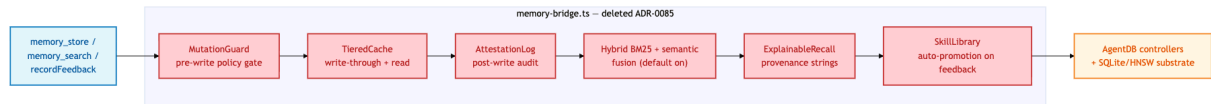
The pattern visible across this timeline:

- **ADR-0053** intended a single-CLI-surface architecture with the bridge as the cross-cutting integration layer.
- **ADR-0084 → ADR-0085** removed the bridge as redundant (true structurally), losing its cross-cutting features (collateral, not yet noticed).
- **ADR-0086** intended a single substrate (RVF-First); failed at Debt 15; **accepted** a dual-substrate residue.
- **ADR-0112 promoted** the accepted residue from "internal substrate detail" to "deliberate dual MCP surface."
- **ADR-0179 (round 1)** discovered the deletion cost. **Round 2** discovered the dual-surface cost.

Each ADR was locally correct. The composition is what produced the current tax.

7. What the bridge did — and what got lost when it died

`memory-bridge.ts` was ADR-0053's "Phase 1 Foundation." It wasn't just a wrapper — it was the single chokepoint where cross-cutting concerns (policy, audit, cache, fusion) attached to every write and read.



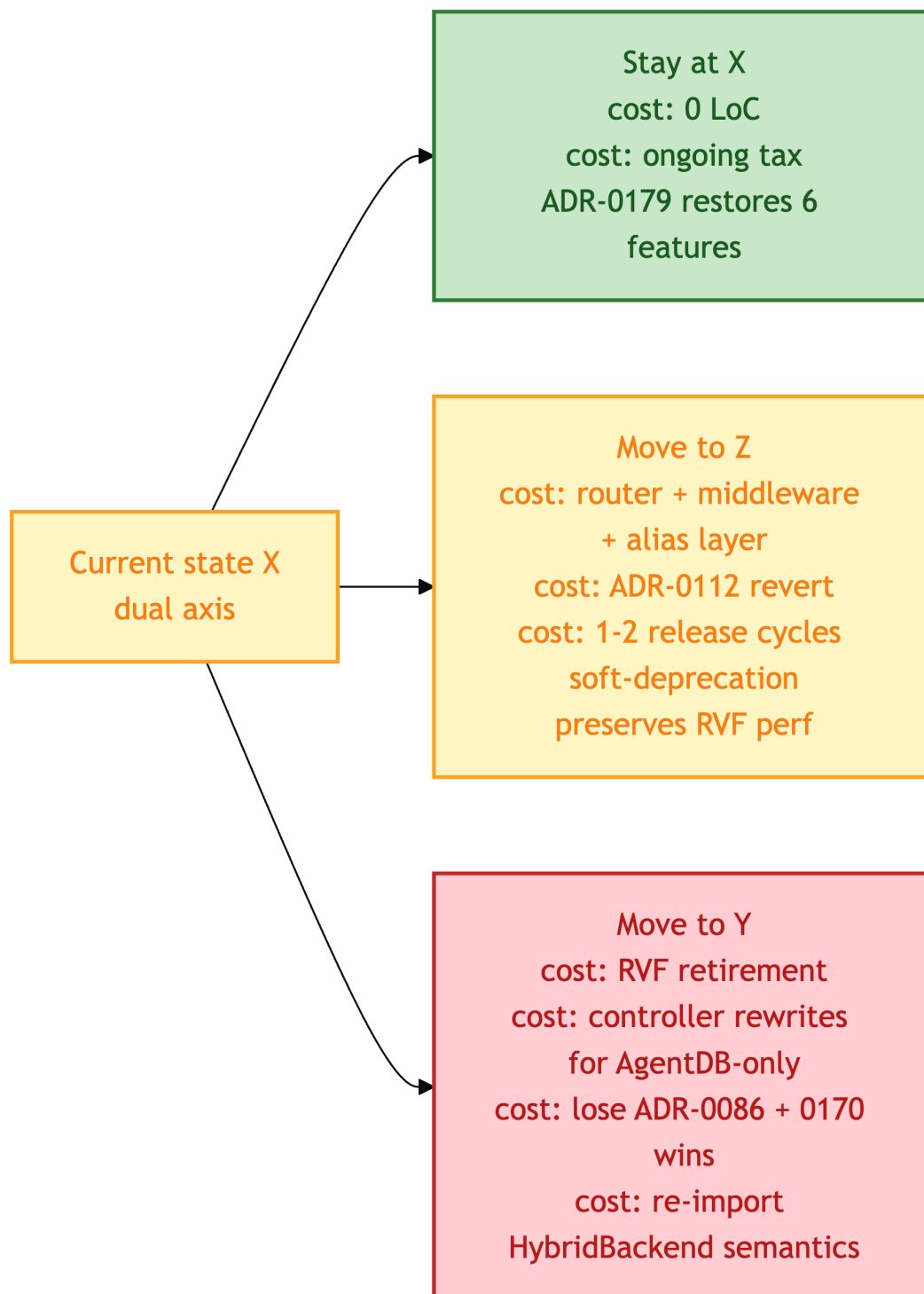
► Mermaid Source

When ADR-0085 deleted the bridge module, all six middleware concerns went with it. The fork's dual axis then made re-wiring them harder, because each is **cross-cutting** — they apply to mutations regardless of substrate. Under upstream's single axis, each had ONE call site to live at. Under the fork's dual axis, each needs TWO call sites (or a shared middleware module imported by both routers).

ADR-0179 is the proposed restoration; ADR-0179 round-1 council ratified 6-B-amended (axis-correct call-site restoration with shared middleware). The bridge does not return; the features do.

8. Migration cost per architectural option

If the fork chose to move from X (current) to Y or Z, what does each transition cost?



► Mermaid Source

- **X (status quo)**: zero migration cost but ongoing tax (cognitive load, doubled ops, ADR churn) and unresolved ADR-0053 regression.
- **Z (single surface, dual substrate)**: moderate cost — router module + shared middleware + compatibility alias layer for `agentdb_*` tools (1-2 release cycles soft-deprecation per

integration-judge). Preserves RVF perf wins. Reverts ADR-0112; amends ADR-0086 to honor Debt 15 honestly.

- **Y (collapse to single-axis)**: largest cost. Retires RVF as primary substrate, migrates all `memory_*` data into AgentDB SQLite, gives up ADR-0086 + ADR-0170 Phase C.3 wins, re-imports HybridBackend's design from upstream. Perf judgment: forfeits Phase C.3, loses 2-3x on hot path unless internally re-architected.

9. Open questions and unknowns

The round-2 devil's advocate flagged that **two pre-flight artifacts must exist before any X/Y/Z vote is meaningful**:

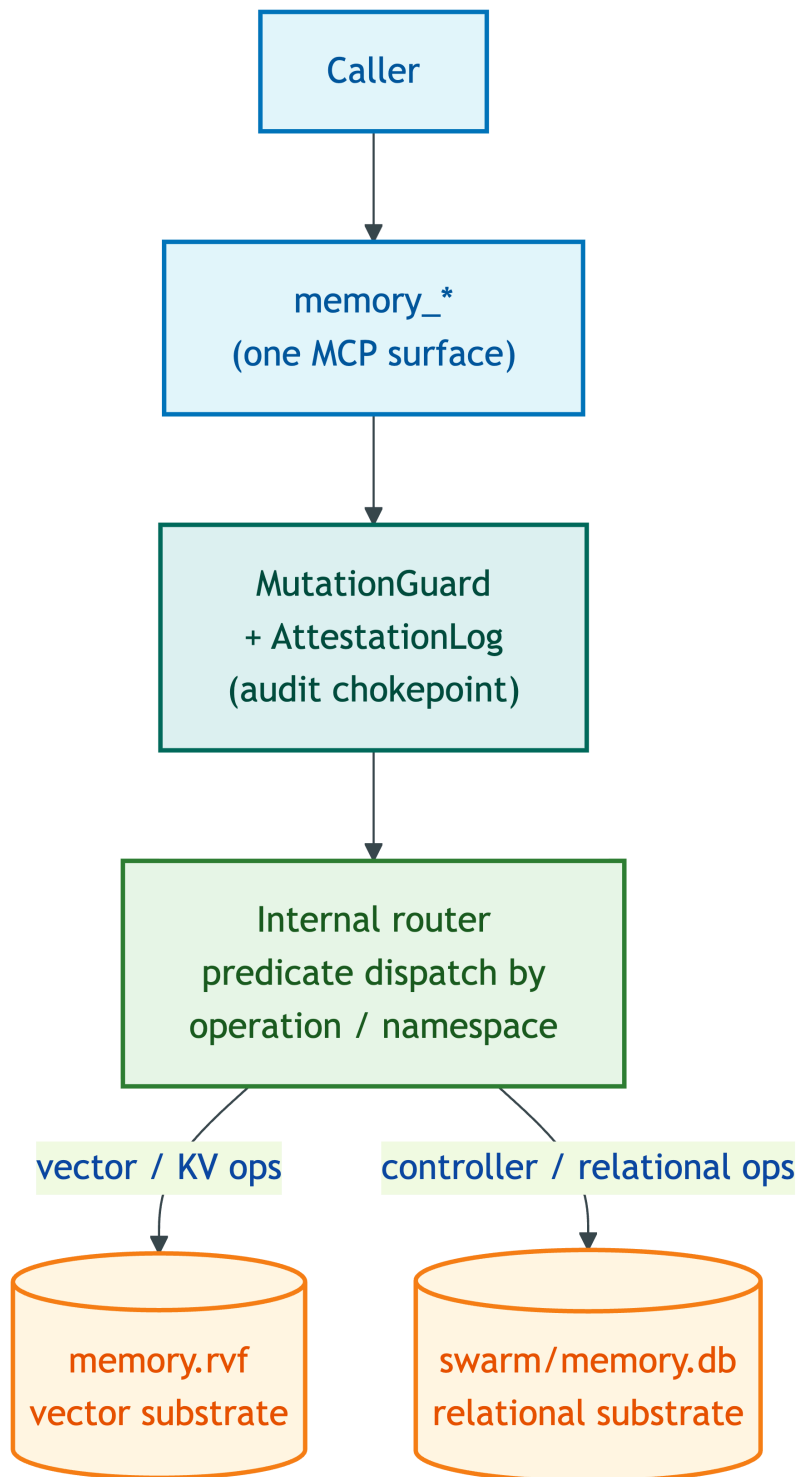
1. **The honest perf benchmark has not been run.** RVF NAPI HNSW vs AgentDB's own native HNSW on representative fork workload — insert, k-NN search, k-NN with metadata JOIN, cold start. Without this measurement, every option is being evaluated against a phantom baseline. Round-2 perf-judge estimated 2-3x but flagged this is *reasoned from code paths*, not measured.
2. **ADR-0086's Status section has not been amended to acknowledge Debt 15 as a partial unification failure.** Currently the ADR title says "Single Storage Abstraction (RVF-First)" while the outcome is dual-storage. Until ADR-0086 is honest about what it delivered vs intended, decisions referencing it inherit the mismatch.

Other unverified claims in this document:

- The "~~3-10ms~~" / "8-25ms" cold-start estimates are reasoned from file-open + WAL-init costs on macOS/SSD; not measured on the fork's test corpus.
- The "~50 agentdb_* tools" count is approximate; the actual count varies by enabled controllers per `agentdb_health`.
- The "9+ ADRs of axis-related decisions" is the count of ADRs that explicitly cite the axis as load-bearing; the actual touchpoint count across all ADRs is higher.

10. The third option (Option Z) — not currently shipped, but unifies the trade-offs

The round-2 council (`docs/council/ADR-0179-council-r2-axis-architecture.md`) concluded 6-of-8 in favor of a synthesis that preserves the fork's substrate decision while reverting the surface split:



► Mermaid Source

Why the council favored Z: substrate split is real and forced (workload bimodality, ~6 RVF-natural / ~6 SQLite-natural features per memory-judge); surface split is optional and over-extended in ADR-0112. Z preserves the perf win of native HNSW (within 1% of current X per perf-judge), eliminates the cognitive tax of dual surface, restores upstream's MCP design intent (single `memory_*` family), and provides a natural home for cross-cutting features at the audit

boundary above the router. ADR-0086 stands at the substrate; ADR-0112 is reverted at the surface.

11. References

Upstream agentdb ADRs (verified 2026-05-12 against `ruvnet/agentdb` HEAD `a478ab3`)

- `ruvnet/agentdb/README.md` — "Single-file cognitive container — vectors, indexes, learning state, and a cryptographic audit trail in one `.rvf`."
- **Upstream ADR-002** — RuVector WASM Complete Integration (~22% implemented per audit; 8 "New Files" don't exist).
- **Upstream ADR-003** — RVF as v3 format (~80% implemented; status field stale).
- **Upstream ADR-004** — AGI Capabilities Integration (~90% implemented).
- **Upstream ADR-005** — Self-Learning Pipeline Integration (**100% implemented** — 6 components, ~2,841 LoC).
- **Upstream ADR-006** — Unified Self-Learning RVF Integration (~70% implemented; **wrapper exists but factory orphaned**; explicit no-pgvector mandate verified in code).
- **Upstream ADR-007** — @ruvector Full Capability Integration (~45% implemented; 5-phase 16-week roadmap to 95% coverage).
- **Upstream ADR-008** — @agentdb/chat (0% implemented; README claim misattributed).
- **Upstream ADR-009** — Causal Atlas RVF Runtime (0% — **6-byte placeholder file "iweds"**).
- **Upstream ADR-010** — @ruvector/rvf-solver v0.1.6 Deep Integration (~70%; 4 MCP solver tools missing).
- `ruvnet/RuVector` **ADR-029** — RVF canonical format; `ruvector-postgres` notably absent from migration table.

Fork ADRs (sparkleideas)

HTML versions of the load-bearing ADRs are at `docs/adr/export/html/` (links below).

- **ADR-0039** ([source](#)) — Upstream AgentDB Controller Integration Roadmap (2026-03-15, superseded). Identified 31 additional upstream classes that could serve as standalone controllers; predecessor of the integration work that produced today's surface.
- **ADR-0067 — HTML** ([source](#)) — The Original Vision: How Controller Wiring Was Supposed to Work (2026-04-05, Closed/informational). Synthesizes the intended two-tier model

(AgentDB Tier 1 + CLI-layer Tier 2); explains the duplicate-controller problem that motivated the bridge work.

- **ADR-0084 — HTML** ([source](#)) Phase 4 (2026-04-12) — Controller-direct access via `getController()` .
- **ADR-0085 — HTML** ([source](#)) (2026-04-13) — Bridge deletion. ~3,650 lines removed.
- **ADR-0086 — HTML** ([source](#)) (2026-04-14) — Layer 1 storage abstraction (RVF-First). Accepted Debt 15 dual-backend.
- **ADR-0112 — HTML** ([source](#)) (2026-05-01) — Independent stores rule. Surface-split formalized.
- **ADR-0170** Phase C.3 — Init-cost split. (Most of ADR-0170's substrate work superseded by ADR-0177.)
- **ADR-0174 + ADR-0175** — Graph axis + ruvector-postgres direction (both superseded by ADR-0177 on 2026-05-12).
- **ADR-0177 — HTML** ([source](#)) (2026-05-12, **proposed**) — **Adopt upstream agentdb's RVF-first single-file Cognitive Container vision.** Supersedes ADR-0170/0174/0175. References-upstream block lists all 11 upstream source documents consulted. Contains the 9-agent implementation audit that drives §2 of this document.
- **ADR-0178** — Restore hierarchical-memory implementation.
- **ADR-0179 — HTML** ([source](#)) (2026-05-13, **proposed**) — Restore controller instrumentation lost in ADR-0085 bridge deletion.

Council transcripts

- **[[ADR-0179-council-r1-bridge-deletion]]** — Should ADR-0085 be undone?
- **[[ADR-0179-council-r2-axis-architecture]]** — Should the axis split itself be undone?

Key memories

- `project-rvf-primary.md` — RVF is primary for `memory_*` ; SQLite is primary for `agentdb_*` (axis-separation per ADR-0166 Amendment).
- `project-fork-only-controllers.md` — 8 fork-only controllers restored from `bd760f2` , 4 deferred.
- `reference-embedding-model.md` — `Xenova/all-mpnet-base-v2` , 768-dim, HNSW `m=23/efC=100/efS=50`.